```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// -------- STRUCTURE --------
struct Item
{
    int id;
    char name[50];
    char colour[30];
    char location[50];
    char type[10]; // Lost / Found
    struct Item *next;
};

struct Item *head = NULL;

// -------- STACK --------
struct Stack
{
    struct Item *data;
    struct Stack *next;
};

struct Stack *top = NULL;

// -------- HELPER: remove newline --------
void removeNewline(char str[])
{
```

```c
void removeNewline(char str[])
    str[strcspn(str, "\n")] = '\0';
}

// -------- STACK PUSH --------
void push(struct Item *newItem)
{
    struct Stack *node = (struct Stack *)malloc(sizeof(struct Stack));
    node->data = newItem;
    node->next = top;
    top = node;
}

// -------- DISPLAY STACK --------
void displayStack()
{
    struct Stack *temp = top;

    if (temp == NULL)
    {
        printf("\nNo recent items.\n");
        return;
    }

    printf("\n--- Recent Items (Stack) ---\n");

    while (temp != NULL)
    {
        printf("\nName: %s | Colour: %s | Location: %s | Type: %s\n",
```

```
                temp->data->name,
                temp->data->colour,
                temp->data->location,
                temp->data->type);
            temp = temp->next;
        }
    }

    // -------- ADD LOST --------
    void addLostItem()
    {
        struct Item *newItem = (struct Item *)malloc(sizeof(struct Item));

        printf("\nEnter ID: ");
        scanf("%d", &newItem->id);
        getchar();

        printf("Enter Item Name: ");
        fgets(newItem->name, 50, stdin);
        removeNewline(newItem->name);

        printf("Enter Colour: ");
        fgets(newItem->colour, 30, stdin);
        removeNewline(newItem->colour);

        printf("Enter Location: ");
        fgets(newItem->location, 50, stdin);
```

```c
    removeNewline(newItem->location);

    strcpy(newItem->type, "Lost");

    newItem->next = head;
    head = newItem;

    push(newItem);

    printf("\nLost item added successfully!\n");
}

// -------- ADD FOUND --------
void addFoundItem()
{
    struct Item *newItem = (struct Item *)malloc(sizeof(struct Item));

    printf("\nEnter ID: ");
    scanf("%d", &newItem->id);
    getchar();

    printf("Enter Item Name: ");
    fgets(newItem->name, 50, stdin);
    removeNewline(newItem->name);

    printf("Enter Colour: ");
    fgets(newItem->colour, 30, stdin);
    removeNewline(newItem->colour);
```

```c

    printf("Enter Location: ");
    fgets(newItem->location, 50, stdin);
    removeNewline(newItem->location);

    strcpy(newItem->type, "Found");

    newItem->next = head;
    head = newItem;

    push(newItem);

    printf("\nFound item added successfully!\n");
}

// -------- DISPLAY --------
void displayItems()
{
    struct Item *temp = head;

    if (temp == NULL)
    {
        printf("\nNo items available.\n");
        return;
    }

    printf("\n--- All Items ---\n");

```

```
129  void displayItems()
141      while (temp != NULL)
142      {
143          printf("\nID: %d", temp->id);
144          printf("\nName: %s", temp->name);
145          printf("\nColour: %s", temp->colour);
146          printf("\nLocation: %s", temp->location);
147          printf("\nType: %s\n", temp->type);
148
149          temp = temp->next;
150      }
151  }
152
153  // -------- MATCH --------
154  void matchItems()
155  {
156      struct Item *i, *j;
157      int found = 0;
158
159      for (i = head; i != NULL; i = i->next)
160      {
161
162          if (strcmp(i->type, "Lost") != 0)
163              continue;
164
165          for (j = head; j != NULL; j = j->next)
166          {
167
168              if (strcmp(j->type, "Found") != 0)
```

```c
                continue;

            if (strcmp(i->name, j->name) == 0 &&
                strcmp(i->colour, j->colour) == 0 &&
                strcmp(i->location, j->location) == 0)
            {

                printf("\nMATCH FOUND!\n");
                printf("Item: %s | Colour: %s | Location: %s\n",
                        i->name, i->colour, i->location);
                printf("Lost ID: %d | Found ID: %d\n", i->id, j->id);

                found = 1;
            }
        }
    }

    if (!found)
        printf("\nNo matches found.\n");
}

// -------- MAIN --------
int main()
{
    int choice;

```

```c
    while (1)
    {
        printf("\n===== Lost and Found System =====\n");
        printf("1. Report Lost Item\n");
        printf("2. Report Found Item\n");
        printf("3. Display Items\n");
        printf("4. Match Items\n");
        printf("5. Show Recent (Stack)\n");
        printf("6. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            addLostItem();
            break;
        case 2:
            addFoundItem();
            break;
        case 3:
            displayItems();
            break;
        case 4:
            matchItems();
            break;
        case 5:
            displayStack();
```

```
            break;
        case 6:
            exit(0);
        default:
            printf("\nInvalid choice!\n");
        }
    }
}
```